

Banco de Dados NoSql

Professor Msc. Aparecido Vilela Junior
aparecido.vilela@unicesumar.edu.br

Apresentação

NoSQL

Problemas que levaram a utilização do NoSQL

Quando utilizar NoSQL?

Tipos

Orientação à documentos

MapReduce

JSON

MongoDB

Quando surgiu

Na criação do termo, existia ausência do SQL e o modelo ainda centrado no relacional

O movimento NoSQL "é completamente distinto do modelo relacional e portanto deveria ser chamado "NoREL" ou algo que produzisse o mesmo efeito".

Carlo Strozzi

Termo NoSQL vira abreviação de Not Only SQL

NoSQL

O que é NoSQL hoje?

Not Only SQL (Não apenas, não somente SQL)

NoSQL = Não Relacional

Atualmente é uma alternativa ao modelo relacional

NoSQL

Problemas que motivaram a utilização do NoSQL

BigData

Volume

Variedade

Velocidade dos Dados

Modelo relacional perde desempenho quando utiliza BigData

NoSQL

Problemas que motivaram a utilização do NoSQL

BigData

Volume

Variedade

Velocidade dos Dados

Modelo relacional perde desempenho quando utiliza BigData

Relacional x NoSQL: a Ponte

Você já aprendeu a modelar dados no paradigma relacional (Banco de Dados I/II). Vamos usar isso como ponte para o NoSQL:

Modelo Relacional (o que você já sabe)	NoSQL / MongoDB (o que vamos aprender)
Tabela	Coleção (collection)
Linha / registro	Documento (JSON/BSON)
Coluna	Campo (field)
Chave primária (PK)	Campo <code>_id</code>
Chave estrangeira + JOIN	Referência ou dado embutido (embedding)
Schema fixo, definido via DDL	Schema flexível (schema-less)
Normalização (evitar redundância)	Desnormalização proposital (otimizar leitura)
Consistência ACID entre tabelas	Consistência por documento + modelo BASE

A pergunta muda de “como evito duplicar dados?” para “quais dados eu preciso ler juntos, com que frequência?”

Volume

O volume refere-se à quantidade total de dados armazenados e processados.

Netflix:

Armazena mais de 60 **petabytes** de dados apenas para seu sistema de análise.

Coleta dados de mais de **270 milhões de assinantes em todo o mundo**, registrando cada clique, pausa, busca e avaliação para alimentar seus algoritmos de recomendação.

Amazon:

Gerencia **um catálogo com centenas de milhões de produtos**.

Processa dados de mais de **300 milhões de clientes ativos**.

A AWS, seu braço de nuvem, armazena uma quantidade de dados na casa dos exabytes (1 exabyte = 1.000 petabytes).

Waze / Google Maps:

Processa dados de localização de mais de 1 bilhão de usuários ativos mensais (Google Maps). O **Waze**, sozinho, **tem mais de 140 milhões**. O volume de dados de geolocalização gerado é contínuo e massivo.

Velocidade

A velocidade é sobre o quão rápido os dados chegam e precisam ser processados.

Serviços Financeiros (Detecção de Fraude):

Grandes processadoras de cartão de crédito, como a Visa, são capazes de processar mais de **65.000 transações por segundo**. Cada uma dessas transações precisa ser analisada em tempo real para detecção de fraude.

Indústria (Manutenção Preditiva - IoT):

Uma única turbina de avião da General Electric pode gerar mais **de 5.000 pontos de dados por segundo**, resultando em terabytes de dados em um único voo.

Netflix:

A plataforma precisa processar em **tempo real os dados de streaming de milhões de usuários** simultâneos para monitorar a qualidade do serviço e identificar problemas de performance instantaneamente.

Relacional x NoSql

A necessidade de processar dados em tempo real, seja para leitura ou escrita, colide com duas características fundamentais do modelo relacional: a garantia de consistência (**ACID**) e a necessidade de **JOINS**.

Velocidade de Escrita (Ingestão de Dados):

O Caso dos Serviços Financeiros e IoT: Uma processadora de cartões que analisa **65.000 transações por segundo** ou **uma turbina de avião que gera 5.000 pontos de dados por segundo** precisam de uma capacidade de escrita (ingestão) massiva.

No Modelo Relacional: Cada INSERT em um banco de dados relacional é uma operação "cara". O banco precisa verificar chaves primárias, chaves estrangeiras, constraints, atualizar índices e garantir as propriedades ACID. Em alta velocidade, esses mecanismos de verificação e bloqueio se tornam um gargalo gigantesco, e o banco simplesmente não consegue acompanhar o ritmo.

Por que falha: A prioridade do modelo relacional é a **consistência**, o que o torna mais lento para escritas. Sistemas de Big Data, muitas vezes, priorizam a disponibilidade e a velocidade de ingestão, aceitando uma "consistência eventual" (o modelo BASE).

Relacional x NoSql

Velocidade de Leitura (Consultas):

O Caso dos Sistemas de Recomendação (Netflix/Spotify): Para recomendar uma música, o Spotify precisa analisar seu histórico de audição, suas playlists, os artistas que você segue e comparar tudo isso com o comportamento de milhões de outros usuários.

No Modelo Relacional: Essa consulta exigiria **múltiplos e complexos JOINS entre tabelas gigantescas (usuarios, musicas_ouvidas, playlists, artistas_seguidos, etc.)**. Executar um **JOIN** entre tabelas com **bilhões** de linhas em **tempo real** para cada usuário é computacionalmente inviável e lento demais para a experiência do usuário.

Por que falha: **A normalização, que é uma força do modelo relacional para evitar redundância**, se torna uma fraqueza de performance em consultas de larga escala. Modelos **NoSQL, como o de documentos, armazenam os dados relacionados juntos, eliminando a necessidade de JOINS caros**.

A variedade mostra que os dados não são apenas números em tabelas.

Netflix:

Dados Estruturados: Informações de contas e faturamento (em bancos de dados relacionais).

Dados Semi-estruturados: Logs de atividade do usuário, eventos de play/pause (armazenados em formatos como JSON).

Dados Não-estruturados: Os próprios arquivos de vídeo e áudio (o conteúdo), imagens de pôsteres, arquivos de legenda.

Saúde (Análise Preditiva):

Estruturados: Dados demográficos de pacientes (idade, sexo).

Semi-estruturados: Prontuários eletrônicos em formatos como HL7 ou XML.

Não-estruturados: Imagens de exames (raios-X, ressonâncias), anotações de médicos em texto livre, dados de sensores de dispositivos vestíveis (smartwatches).

Amazon:

Estruturados: Dados de vendas, estoque, preços.

Semi-estruturados: Dados de navegação do site (clickstream).

Não-estruturados: Avaliações de produtos (texto), imagens e vídeos dos produtos.

O Caso da Netflix e Amazon:

A Netflix armazena **petabytes de dados sobre o comportamento de seus 270 milhões de assinantes**. A Amazon processa dados de **mais de 300 milhões de clientes**.

No Modelo Relacional: Tentar armazenar todos os cliques, visualizações e compras em um único banco de dados relacional (mesmo um cluster) seria financeiramente inviável e tecnicamente impossível. O **custo de um único supercomputador capaz de lidar com esse volume seria astronômico**. Além disso, a performance de consultas em tabelas com trilhões de linhas degradaria a ponto de se tornar inútil.

Por que falha: A arquitetura relacional **não** foi nativamente projetada para ser **distribuída massivamente**. Embora existam técnicas de "sharding" (particionamento) para bancos relacionais, elas são complexas de implementar e gerenciar, enquanto em sistemas como o MongoDB, o sharding é uma característica central.

Relacional x NoSql

O Caso da Netflix e Amazon:

A Netflix armazena **petabytes de dados sobre o comportamento de seus 270 milhões de assinantes**. A Amazon processa dados de **mais de 300 milhões de clientes**.

No Modelo Relacional: Tentar armazenar todos os cliques, visualizações e compras em um único banco de dados relacional (mesmo um cluster) seria financeiramente inviável e tecnicamente impossível. O **custo de um único supercomputador capaz de lidar com esse volume seria astronômico**. Além disso, a performance de consultas em tabelas com trilhões de linhas degradaria a ponto de se tornar inútil.

Por que falha: A arquitetura relacional **não** foi nativamente projetada para ser **distribuída massivamente**. Embora existam técnicas de "sharding" (particionamento) para bancos relacionais, elas são complexas de implementar e gerenciar, enquanto em sistemas como o MongoDB, o sharding é uma característica central.

60 segundos???

Estimativa do que acontece a cada minuto na internet (dados de dezembro/2025):

- E-mails enviados: 347 mil
- Mensagens no WhatsApp: 41 milhões
- Buscas no Google: 6 milhões
- Interações no Facebook: 1,7 milhão
- Stories criados no Instagram: 695 mil
- Vídeos assistidos no TikTok: 167 milhões
- Vídeo enviado ao YouTube: 500 horas de conteúdo

Tudo isso precisa ser armazenado, replicado e consultado em milissegundos — é aqui que o NoSQL entra.

Fontes: Domo “Data Never Sleeps” 2025; Statista; DataReportal (estimativas, margem de $\pm 10\text{--}20\%$).

Big Data em Números

500 h

de vídeo enviadas
ao YouTube por minuto

167 M

vídeos assistidos
no TikTok por minuto

41 M

mensagens enviadas
no WhatsApp por minuto

6 M

buscas realizadas
no Google por minuto

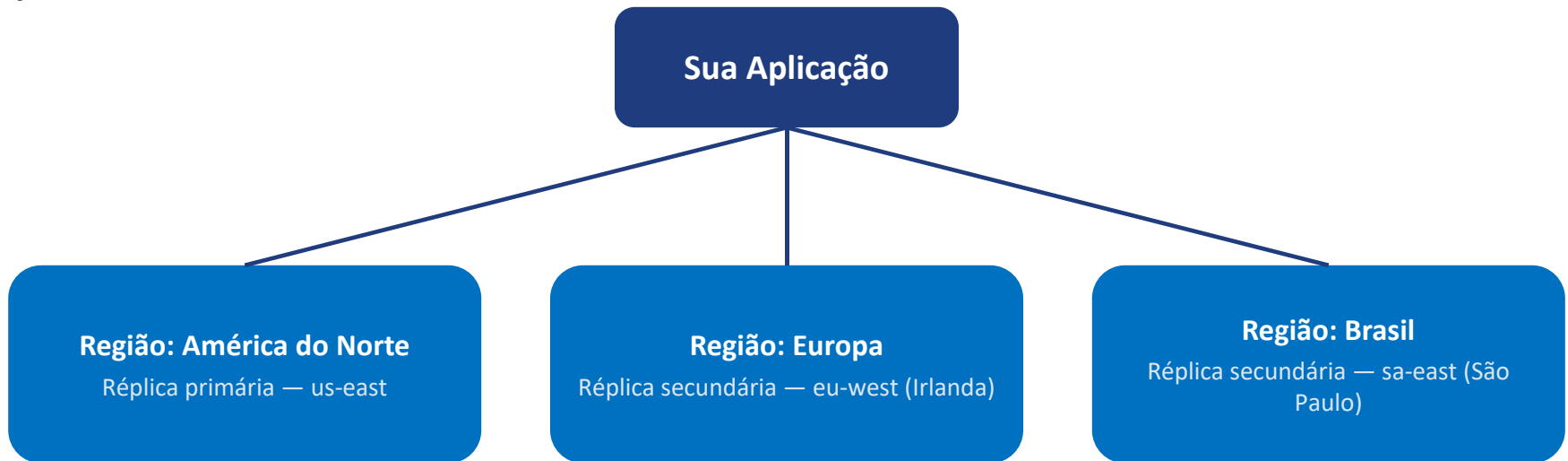
≈ 1,74 milhão de GB (1,74 petabyte) de dados criados e consumidos por minuto, no mundo todo

≈ 181 zettabytes gerados globalmente em 2025 (1 zettabyte = 1 bilhão de terabytes)

Nenhum banco relacional tradicional, rodando em um único servidor, sustenta esse volume e essa velocidade de escrita/leitura simultâneas. É exatamente esse cenário que motiva os bancos NoSQL: distribuir os dados entre várias máquinas (escalabilidade horizontal).

NoSQL na Nuvem

Um mesmo cluster NoSQL pode manter réplicas dos dados em data centers de continentes diferentes, funcionando como um único banco:



Latência menor

os dados ficam fisicamente mais perto do usuário final.

Alta disponibilidade

se uma região cair, outra réplica assume o atendimento.

Soberania de dados

atende exigências legais como a LGPD (Brasil) e o GDPR (UE).

Isso só é possível porque o modelo NoSQL foi pensado, desde o início, para escalar horizontalmente e tolerar a latência entre servidores distantes (ver Teorema CAP, a seguir).

ACID vs BASE

ACID (Relacional): Atomicidade, Consistência, Isolamento, Durabilidade

BASE (NoSQL): Basicamente Disponível, Estado Flexível, Consistência Eventual

 ACID: Transação bancária

 BASE: Redes sociais e sistemas de grande escala

 Compromisso entre consistência forte e escalabilidade

ACID vs BASE

ACID é o conjunto de garantias que os bancos de dados relacionais (como PostgreSQL, MySQL, SQL Server) tradicionalmente oferecem.

A prioridade máxima é a consistência e a confiabilidade dos dados.

Cada transação é tratada com o máximo rigor, como uma operação em um cofre de banco.

ACID vs BASE

A - Atomicidade (Atomicity):

O que é: "Tudo ou nada". Uma transação é uma unidade de trabalho indivisível. Ou todas as operações dentro dela são concluídas com sucesso, ou nenhuma delas é. Não existe meio-termo.

Exemplo Prático: Em uma transferência de daContaA para a ContaB, duas coisas precisam acontecer : o débito da Conta A e o crédito de na Conta B. Se o sistema falhar depois do débito, mas antes do crédito, a atomicidade garante que a operação de débito seja desfeita (ROLLBACK). O dinheiro nunca fica "no limbo".

ACID vs BASE

C - Consistência (Consistency):

O que é: A transação leva o banco de dados de um estado válido para outro estado válido. Todas as regras do banco de dados (constraints, chaves estrangeiras, etc.) são sempre respeitadas.

Exemplo Prático: O saldo total do banco deve ser consistente. A transferência não "cria" nem "destrói" dinheiro; ela apenas o move. Se uma regra de negócio diz que uma conta não pode ficar com saldo negativo, uma transação que viole essa regra será bloqueada e desfeita.

ACID vs BASE

I - Isolamento (Isolation):

O que é: Transações que ocorrem ao mesmo tempo não devem interferir umas nas outras. Do ponto de vista de uma transação, parece que ela está sendo executada sozinha no banco de dados.

Exemplo Prático: Se você tem R\$ 500 na conta e tenta fazer duas transferências de R\$ 400 ao mesmo tempo em caixas eletrônicos diferentes, o isolamento garante que apenas uma delas será bem-sucedida. As transações são colocadas em uma "fila" invisível, impedindo que o saldo final fique incorreto.

ACID vs BASE

D - Durabilidade (Durability):

O que é: Uma vez que a transação é confirmada (COMMIT), ela é permanente e sobreviverá a qualquer falha subsequente, como uma queda de energia ou um travamento do servidor.

Exemplo Prático: Assim que o banco confirma que sua transferência foi concluída, você pode ter certeza de que o dinheiro chegou ao destino e que essa operação não será desfeita, mesmo que o sistema caia um segundo depois.

ACID vs BASE

O Modelo BASE (A "Rede Social")

BASE é uma filosofia adotada por muitos sistemas NoSQL (como MongoDB, Cassandra, DynamoDB) projetados para escala massiva.

Aqui, a prioridade máxima é a disponibilidade e a escalabilidade, mesmo que isso signifique sacrificar a consistência imediata.

ACID vs BASE

BA - Basicamente Disponível (Basically Available):

O que é: O sistema garante que estará disponível para responder a uma requisição. Ele não vai travar ou dar um erro, mas pode responder com dados ligeiramente desatualizados. A disponibilidade é mais importante que a consistência.

Exemplo Prático: Você posta uma foto no Instagram. O sistema sempre estará disponível para mostrar seu perfil para seus amigos, mesmo que a foto nova ainda não tenha aparecido para um amigo no Japão. O sistema responde, mesmo que não seja com a informação mais recente para todos.

ACID vs BASE

S - Estado Flexível (Soft State):

O que é: O estado do sistema pode mudar ao longo do tempo, mesmo sem uma nova entrada. Os dados são considerados "flexíveis" ou "em fluxo" até que atinjam a consistência.

Exemplo Prático: Sua foto recém-postada recebe uma curtida. O contador de curtidas pode mostrar "1" para você nos EUA, mas ainda mostrar "0" para seu amigo na Europa por alguns segundos. O estado do "contador de curtidas" é flexível e está se propagando pela rede.

ACID vs BASE

E - Consistência Eventual (Eventually Consistent):

O que é: Esta é a chave do modelo BASE. O sistema garante que, se nenhuma nova atualização for feita, todas as réplicas dos dados irão, eventualmente, convergir para o mesmo estado. Não há garantia de quando isso acontecerá, apenas que acontecerá.

Exemplo Prático: Sua foto recebe 100 curtidas. Por um breve período, diferentes usuários em diferentes partes do mundo podem ver 98, 99 ou 100 curtidas. Mas, após alguns segundos ou minutos, todos os servidores convergirão e mostrarão consistentemente 100 curtidas. Para uma rede social, essa pequena inconsistência temporária é perfeitamente aceitável em troca de uma performance e escala globais.

ACID vs BASE

Característica	ACID (Relacional)	BASE (NoSQL)
Prioridade Principal	Consistência e Confiabilidade	Disponibilidade e Escalabilidade
Consistência	Forte e imediata. Os dados estão sempre corretos.	Eventual. Os dados podem ficar temporariamente inconsistentes.
Disponibilidade	Pode ser sacrificada para garantir a consistência (ex: bloqueios de tabela).	Altíssima. O sistema prioriza responder, mesmo com dados desatualizados.
Escalabilidade	Escala verticalmente (servidores mais potentes), mais difícil de distribuir.	Escala horizontalmente (mais servidores), ideal para sistemas distribuídos.
Exemplo de Uso	Sistemas financeiros, ERPs, qualquer sistema onde a precisão é crítica.	Redes sociais, catálogos de e-commerce, sistemas de IoT, Big Data.

ACID vs BASE

A escolha entre ACID e BASE não é sobre "qual é melhor", mas sobre "qual é o mais adequado para o problema que estou resolvendo?".

Para seu dinheiro, você quer as garantias de um cofre (ACID).

Para suas curtidas, você quer a velocidade e a disponibilidade de uma rede global (BASE).

Aplicações Reais do MongoDB

- ⚽ Cartola FC: MongoDB para escalar durante o campeonato
- 🚀 Uber: Armazenamento de dados geográficos com NoSQL
- 🛒 Mercado Livre: Catálogo flexível de produtos com MongoDB
- 📁 NoSQL escolhido por flexibilidade, escalabilidade e performance

No ano 2000 o norte-americano Eric Brewer fez uma apresentação que entrou para a história da ciência da computação e influenciou de maneira marcante a implementação de todo sistema distribuído a partir daquele momento, especialmente os não relacionais.

Foi apresentado o teorema CAP, que prova só ser possível a qualquer sistema distribuído implementar no máximo duas das três características abaixo:

Consistência: todos os nós de um sistema distribuído acessam exatamente a mesma informação a qualquer momento;

Disponibilidade (Availability): toda requisição recebida pelo sistema gerará uma resposta;

Tolerância a falhas (PartitionTolerance): o sistema continua a funcionar caso um ou mais de seus nós enfrente dificuldades.

Teorema CAP - Analogia

"Imaginem o Teorema CAP como planejar uma festa com dois amigos por um telefone com péssima conexão. Vocês precisam decidir três coisas:

Consistência (C): Todos precisam ter a mesma informação. Se a festa é às 20h, todos devem saber que é às 20h, e não um achar que é às 21h.

Disponibilidade (A): O telefone precisa funcionar. Toda vez que você pergunta "Que horas?", você precisa receber uma resposta, mesmo que não seja a mais atualizada.

Tolerância a Partições (P): A 'partição' é a falha na rede. O telefone está picotando. Vocês precisam conseguir continuar planejando mesmo com a linha ruim.

O Teorema diz que, com a linha ruim (P), você não pode ter os outros dois ao mesmo tempo.

Ou vocês pausam a conversa até a linha melhorar para garantir que todos tenham a informação 100% correta (abrindo mão da **Disponibilidade** para ter **Consistência - modelo CP**),

Ou vocês continuam conversando e tomando decisões, aceitando que alguém pode ter uma informação um pouco defasada até a linha melhorar (abrindo mão da **Consistência** para ter **Disponibilidade - modelo AP**)."

Teorema CAP - Exemplo

Sistemas CP (Consistência + Tolerância): Pense em um sistema bancário. Ao fazer uma transferência, é inaceitável que você veja um saldo e o caixa veja outro. O sistema prefere ficar indisponível por alguns segundos a mostrar um dado inconsistente.

Sistemas AP (Disponibilidade + Tolerância): Pense em redes sociais. Se você curte uma foto, não há problema se seu amigo só vir a contagem de "likes" atualizada alguns segundos depois. O mais importante é que o sistema esteja sempre no ar, permitindo que você continue interagindo. O Amazon DynamoDB, usado no carrinho de compras da Amazon, é um exemplo clássico que prioriza a disponibilidade.

MapReduce

MapReduce(Framework desenvolvido pelo Google) - Modelo de programação paralela para processamento largamente distribuído de grandes volumes de dados.

MAP - processo de mapear a requisição do originador.

REDUCE - processo de agregação do resultado em algo consolidado.

MapReduce

Ele foi criado para resolver um problema específico: como processar **volumes de dados** absurdamente grandes (terabytes ou petabytes) de forma rápida e eficiente, usando um cluster de computadores comuns e baratos.

Vamos usar uma analogia para entender o processo: **contar os votos de uma eleição nacional.**

O Problema: **Um único fiscal (um supercomputador)** tentando contar milhões de votos de todas as urnas do país. Seria um processo extremamente lento, caro e, se o fiscal ficasse doente (o servidor falhasse), todo o trabalho pararia.

A Solução MapReduce: Contratar um exército de fiscais (computadores comuns) para trabalhar em paralelo.

A Etapa MAP (Mapear e Organizar)

O que é: A primeira fase, onde o grande problema é quebrado em pedaços menores. Cada "trabalhador" (chamado de Mapper) processa um pequeno pedaço dos dados e o organiza de forma padronizada, emitindo pares de chave-valor.

Na Analogia:

Divisão: As urnas de todo o país são divididas por estado ou cidade.

Mapeamento: Cada fiscal (Mapper) recebe uma única urna. Sua tarefa é simples: abrir a urna, pegar cada voto e colocá-lo em uma pilha separada por candidato. Ao final, ele terá uma pilha para o "Candidato A", uma para o "Candidato B", etc.

Resultado Intermediário: O resultado desta fase não é o total de votos, mas sim milhares de pequenas pilhas de votos, todas pré-organizadas por candidato (a "chave" é o candidato, o "valor" é um voto).

Fase Intermediária: Shuffle and Sort (Embaralhar e Ordenar)

O que é: Esta é uma fase automática e crucial. O sistema pega todos os resultados intermediários dos Mappers e os agrupa pela mesma chave.

Na Analogia: Alguém passa recolhendo todas as pilhas do "Candidato A" de todos os fiscais do país e as junta em um único e gigantesco monte. O mesmo é feito para o "Candidato B", e assim por diante. Agora, em vez de milhares de pequenas pilhas, temos um grande monte de votos para cada candidato, mas ainda não sabemos o total.

A Etapa REDUCE (Reduzir e Agregar)

A segunda fase, onde um "trabalhador" (chamado de Reducer) pega todos os valores associados a uma única chave e produz um resultado final agregado.

Na Analogia:

Agregação: Designamos um "fiscal-chefe" (Reducer) para cada candidato.

Redução: O fiscal-chefe do "Candidato A" recebe o monte gigante com todos os votos para o Candidato A. Sua única tarefa é contar o total. Ele "reduz" a lista enorme de votos a um único número: o total final para aquele candidato.

O mesmo acontece para os outros candidatos, cada um com seu próprio fiscal-chefe.

Resultado Final: Cada Reducer reporta seu total, e temos o resultado completo da eleição de forma massivamente paralela e rápida.

A Etapa REDUCE (Reduzir e Agregar)

A segunda fase, onde um "trabalhador" (chamado de Reducer) pega todos os valores associados a uma única chave e produz um resultado final agregado.

Na Analogia:

Agregação: Designamos um "fiscal-chefe" (Reducer) para cada candidato.

Redução: O fiscal-chefe do "Candidato A" recebe o monte gigante com todos os votos para o Candidato A. Sua única tarefa é contar o total. Ele "reduz" a lista enorme de votos a um único número: o total final para aquele candidato.

O mesmo acontece para os outros candidatos, cada um com seu próprio fiscal-chefe.

Resultado Final: Cada Reducer reporta seu total, e temos o resultado completo da eleição de forma massivamente paralela e rápida.

Quando utilizar NoSQL?

Escalabilidade

Flexibilidade

Manipulação de quantidade massiva de dados

Bom desempenho

Facilidade para consultas

NoSQL

Tipos de NoSQL

Key/Value: DynamoDb, Riak, Azure Table Storage, Berkeley DB, etc.

Wide column store: Hadoop, Cassandra, Hypertable, Amazon SimpleDB, etc.

Document store: MongoDB, CouchDB, RavenDb, etc.

Graph store: Neo4J, Infinite Graph, InforGrid, HyperGraphDB, etc.

NoSQL



Orientação à documentos

Orientação à documentos

Definição

Segundo Anderson (Anderson, et al., 2009), BDODs utilizam o conceito de **dados** e documentos **autocontidos** e **auto descritivos**, isso implica que o documento em si já define como ele deve ser apresentado e o significado dos dados em cuja sua estrutura estão armazenados.

Exemplo: Uma nota fiscal.

Orientação à documentos

Características

Permite que tenha **redundância e inconsistência**.

Contêm **todas** as informações importantes em um único documento.

Livre de esquemas.

Identificadores Únicos Universais(UUID)

Consultar os documentos através de métodos avançados de **agrupamento e filtragem: MapReduce**

Orientação à documentos

JSON

É um padrão leve de intercâmbio de dados, projetado para **facilitar a leitura e escrita** de estruturas, também sendo de **fácil interpretação e geração** por máquinas.

Suas estruturas são baseadas em um subconjunto da especificação da linguagem **JavaScript** e permitem a construção de estruturas de dados partindo de dois conceitos: uma coleção de pares **chave/valor** e uma **lista ordenada de valores** (JSON, 2008).

Orientação à documentos

Formato JSON

```
{  
  nome : Jean carlo Nascimento,  
  email: jnascimento@gmail.com,  
  eventos : [  
    {titulo : Primeiro!},  
    {titulo : Legal!},  
    {titulo : Festao, publicado : true}  
  ]  
}
```

Orientação à documentos

Vantagens

Ganha Flexibilidade, disponibilidade, linguagem de consulta simples e performance.

Desvantagem

Perde em consistência.



MongoDB

Apresentação da ferramenta

Código aberto

Alta performance (desempenho)

Sem esquemas

Orientado à documentos

Escrito em C++

Multiplataforma

É formado por um conjunto de Aplicativos JSON

MongoDB

Criação

Início 2007

1ª versão em 2009

Usado por empresas como Uber, eBay, Spotify e Gov.br

Funcionamento

JSON – estilo de documento com esquemas dinâmicos para oferecer simplicidade e poder.

Suporte de índice completo

Auto-Sharding - escala horizontalmente sem comprometer a funcionalidade.

Consultas Ricas, baseada em documentos

MapReduce - Agregação flexível e processamento de dados.

GridFS - Armazenar arquivos de qualquer tamanho sem complicação.

MongoDB

Analogia à BD Relacional

O documento JSON seria o registro;

A collection seria uma tabela;

Os índices seriam os índices mesmo;

O embedding e o linking seria o join.

MongoDB

Algumas características interessantes do MongoDB

É possível acrescentar dados, por meio de um update;

Não existe um schema dentro das coleções, ou seja, as collections podem ser dinâmicas;

Há upserts, que podem atualizar se documento já existe, ou cria se ele não existe;

Você pode armazenar o seu dado da forma que ele ocorre na “natureza”.

MongoDB

Vantagens do MongoDB

-  Armazenamento orientado a documentos
-  Alta escalabilidade horizontal (sharding)
-  Suporte a replicação e failover
-  Alta performance em leitura e escrita
-  Esquema flexível — ideal para dados variáveis

Alta escalabilidade horizontal (sharding)

O que é: Deixar o mesmo servidor mais forte.

Analogia: Comprar um carro com um motor cada vez mais potente. Você troca seu motor 1.0 por um 2.0, depois por um V8, depois por um motor de foguete.



Problemas:

Custo Exponencial: Fica absurdamente caro.

Limite Físico: Chega uma hora que não existe um "motor" mais potente para comprar.

Ponto Único de Falha: Se esse super-servidor falhar, todo o sistema para.

Alta escalabilidade horizontal (sharding) - Dividir para Conquistar

O que é: Distribuir a carga de trabalho entre vários servidores mais simples e baratos.

Analogia: Em vez de um único caminhão gigante para fazer uma mudança, você usa uma frota de 10 vans menores. Se uma van quebrar, as outras continuam o trabalho.

Vantagens:

Custo-Benefício: Usar hardware comum é muito mais barato.

Escalabilidade "Infinita": Precisa de mais capacidade? É só adicionar mais um servidor à frota.

Alta Disponibilidade: A falha de um servidor não derruba o sistema inteiro.

Conceito-Chave: No MongoDB, a principal técnica para escalabilidade horizontal é chamada de Sharding.



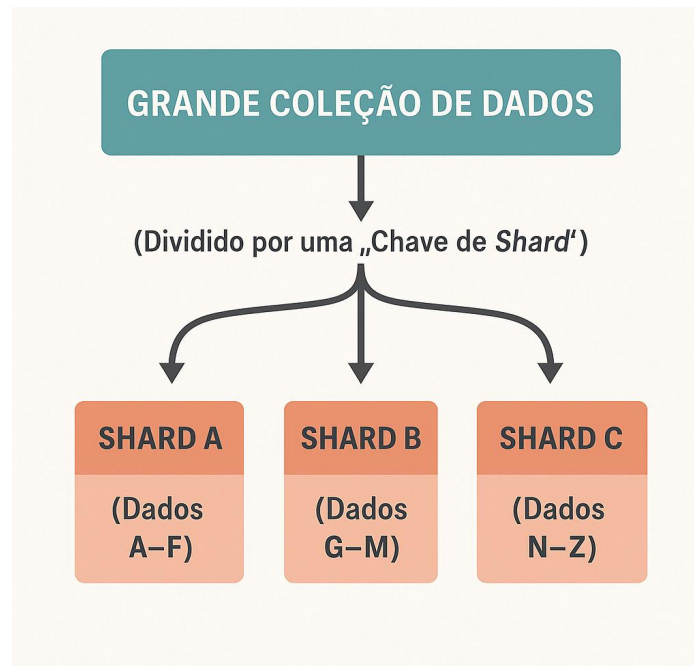
Sharding: Fatiando seus Dados

Definição: Sharding é o processo de dividir uma coleção de dados muito grande em pedaços menores, chamados shards. Cada shard é um banco de dados MongoDB independente que armazena uma porção do total de dados.

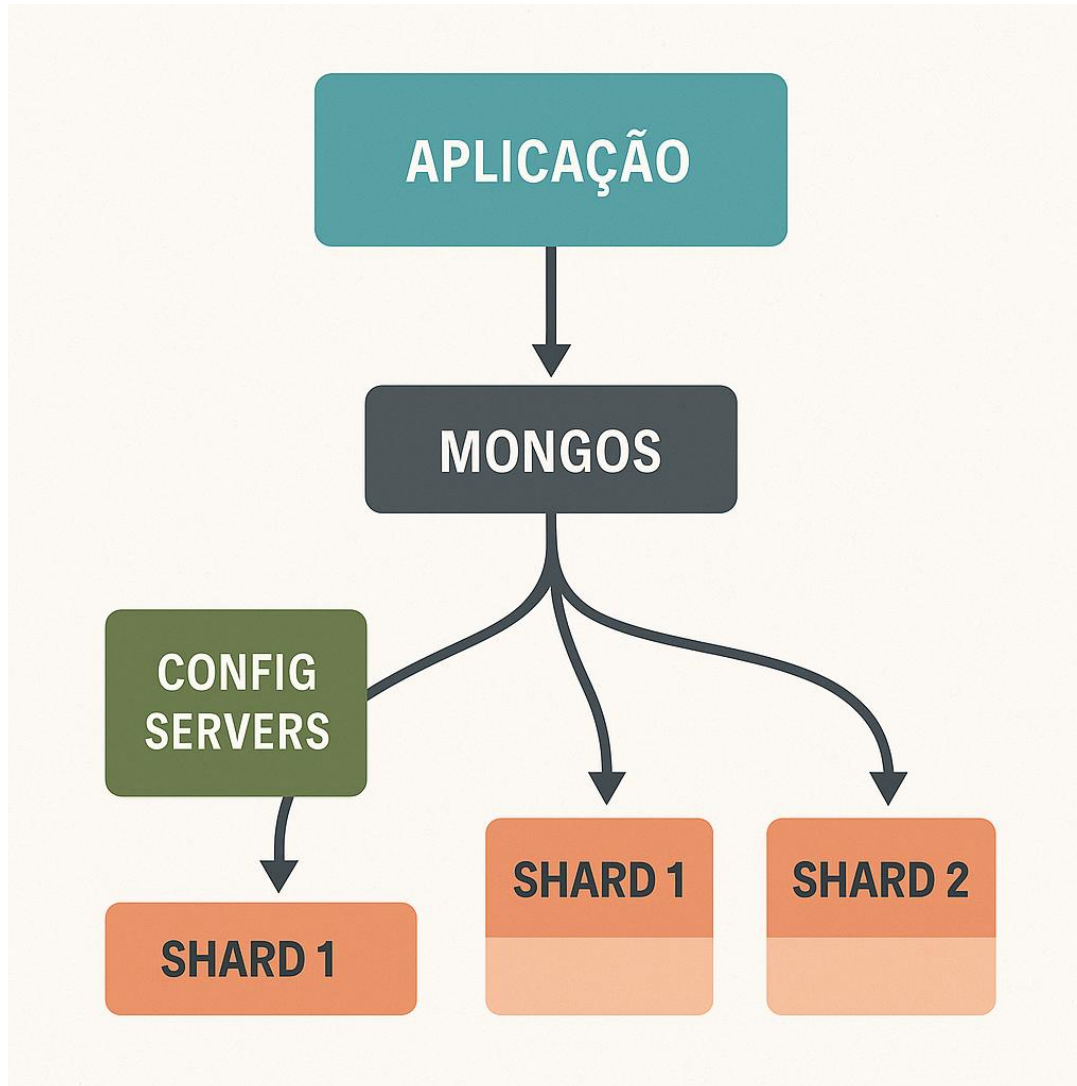
Diagrama Simples:

Uma grande "Coleção de Dados" sendo fatiada em três partes.

Cada fatia vai para um "Shard" (Shard A, Shard B, Shard C).



Os 3 Componentes Essenciais



Os 3 Componentes Essenciais

Shards (Os "Armazéns"):

Função: Armazenar os "pedaços" dos dados.

Detalhe: Cada shard é, na verdade, um replica set (um conjunto de cópias do mesmo banco de dados) para garantir a alta disponibilidade.

Mongos (O "Roteador Inteligente"):

Função: É a porta de entrada para a sua aplicação. Ele recebe as consultas, descobre em qual shard os dados estão e direciona a operação para o lugar certo.

Importante: O Mongos não armazena dados. Ele é apenas um roteador.

Config Servers (O "Mapa do Tesouro"):

Função: Armazenam os metadados do cluster. Eles sabem exatamente qual "pedaço" de dado está em qual "shard".

Analogia: É o "mapa" que o roteador (Mongos) consulta para saber onde encontrar os dados.

Exemplo:

Cenário: Uma loja virtual com uma coleção pedidos que tem bilhões de registros. Consultar os pedidos de um cliente específico está ficando muito lento.

Solução: Fazer sharding na coleção pedidos.

A "Chave de Shard" (Shard Key): Precisamos escolher um campo para dividir os dados.

Péssima escolha: status_pedido. Por quê? Poucos valores ("processando", "pago", "enviado"). Todos os novos pedidos teriam o status "processando" e iriam para o mesmo shard, criando um gargalo (hot shard).

Boa escolha: id_cliente. Por quê? Milhões de valores diferentes. O MongoDB pode distribuir os pedidos de diferentes clientes entre os shards de forma equilibrada.

Exemplo:

Como fica:

Shard A: Armazena pedidos dos clientes 1 a 1.000.000.

Shard B: Armazena pedidos dos clientes 1.000.001 a 2.000.000.

E assim por diante...

Resultado: Quando a aplicação consulta os pedidos do cliente 550.000, o Mongos sabe imediatamente que precisa ir apenas ao Shard A, tornando a consulta muito mais rápida.

Exemplo 2: Sistema de Logs ou IoT

Cenário: Um sistema que coleta milhões de eventos de log ou dados de sensores de IoT por hora. A coleção eventos cresce de forma explosiva.

Solução: Fazer sharding na coleção eventos.

A "Chave de Shard":

Boa escolha: timestamp (a data e hora do evento). O MongoDB distribuiria os eventos ao longo do tempo. Eventos de Janeiro ficariam no Shard A, de Fevereiro no Shard B, etc.

Escolha ainda melhor (Chave Composta): { id_sensor: 1, timestamp: 1 }.

Exemplo 2: Sistema de Logs ou IoT

Como fica:

O MongoDB primeiro agrupa os eventos por `id_sensor` e depois os ordena por timestamp.

Resultado: Isso é extremamente eficiente para consultas que pedem "me mostre todos os eventos do sensor X na última hora".

O Mongos vai diretamente para o shard que contém os dados daquele sensor e busca apenas no intervalo de tempo correto.

Suporte a replicação e failover

Alta Disponibilidade

Cenário: Imagine um sistema crítico, como um aplicativo de delivery, rodando com um único servidor de banco de dados.



Perguntas para reflexão:

O que acontece se o hardware desse servidor queimar?

O que acontece se precisarmos parar o servidor para uma atualização de segurança?

E se a conexão de rede com esse servidor cair?

Resposta: A aplicação para de funcionar. Os clientes não conseguem fazer pedidos, os restaurantes não recebem as demandas. **É o caos.**

Conceito: A replicação é o processo de manter múltiplas cópias idênticas e sincronizadas dos seus dados em diferentes servidores.

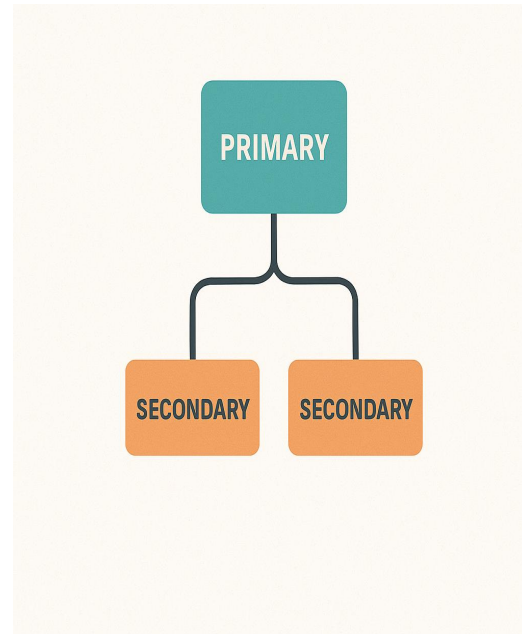
Analogia: É como ter um "dublê" (ou vários) para o seu ator principal. Se o ator principal ficar doente, um dos dublês assume o papel imediatamente, e a peça continua.

****No MongoDB, isso é feito através de um **Replica Set**.**

Conceito: A replicação é o processo de manter múltiplas cópias idênticas e sincronizadas dos seus dados em diferentes servidores.

Analogia: É como ter um "dublê" (ou vários) para o seu ator principal. Se o ator principal ficar doente, um dos dublês assume o papel imediatamente, e a peça continua.

****No MongoDB, isso é feito através de um **Replica Set**.**



Nó PRIMARY (Primário): O ator principal. É o único nó que recebe todas as operações de escrita (INSERT, UPDATE, DELETE). Só existe um Primary por vez em um Replica Set.

Nós SECONDARY (Secundários): Os dublês. Eles recebem uma cópia de todos os dados do Primary. Eles não aceitam escritas, mas podem ser usados para operações de leitura.

Nó ARBITER (Árbitro - Opcional): O juiz. É um nó leve que não armazena dados. Sua única função é votar nas eleições para decidir quem será o novo Primary, ajudando a evitar empates.

O Processo:

A sua aplicação (ex: iFood) envia um novo pedido. A operação de INSERT vai exclusivamente para o nó **PRIMARY**.

O Primary executa a inserção e grava um registro dessa operação em um "diário" especial chamado Oplog (Operation Log).

Os nós **SECONDARY** estão constantemente "assistindo" ao Oplog do Primary.

Assim que uma nova operação aparece no Oplog, os Secondaries a copiam e a aplicam em seu próprio banco de dados.

Resultado: Em milissegundos, todos os nós do Replica Set possuem o novo pedido registrado.

Failover? A Mágica da Alta Disponibilidade

O Processo de Eleição (Failover):

Os nós Secundários percebem que o Primary não responde mais (o "coração" parou de bater).

Eles automaticamente iniciam uma eleição para escolher um novo líder.

O nó Secundário que tiver os dados mais atualizados (verificado pelo Oplog) vence a eleição.

O vencedor se promove a PRIMARY e passa a aceitar as operações de escrita da aplicação.

Resultado: Esse processo é totalmente automático e leva apenas alguns segundos. Para a aplicação, pode parecer uma pequena instabilidade na rede, mas o sistema como um todo continua funcionando.

Manutenção sem Parar o E-commerce

Cenário: A equipe de infraestrutura precisa aplicar uma atualização de segurança urgente no servidor do banco de dados de um e-commerce. A loja não pode sair do ar.

Solução com Replica Set:

Passo 1: A equipe realiza a manutenção em um dos nós Secundários. A aplicação continua funcionando normalmente, escrevendo no Primary.

Passo 2: Após o Secundário voltar, a equipe força uma eleição para que ele se torne o novo Primary.

Passo 3: O antigo Primary agora é um Secundário. A equipe realiza a manutenção nele com segurança, sem impactar a loja.

Resultado: Manutenção crítica realizada com zero downtime para o cliente.

Exemplo Prático

Balanceamento de Carga de Leitura em um Portal de Notícias

Cenário: Um portal de notícias tem um volume gigantesco de leituras (milhões de usuários acessando as notícias) e poucas escritas (jornalistas publicando). O servidor Primary está sobrecarregado com tantas leituras.

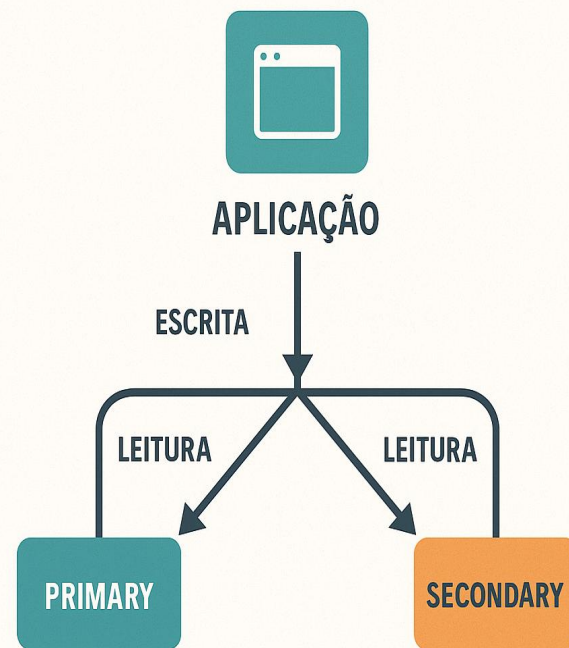
Solução com Replica Set:

Configurar a aplicação para direcionar todas as **escritas** para o **Primary**.

Configurar a aplicação para distribuir as **leituras** entre todos os nós **Secundários**.

Resultado: A carga de trabalho é dividida. O Primary fica livre para as escritas, e os Secundários cuidam da demanda massiva de leituras, melhorando a performance e a velocidade de resposta do site.

LIDANDO COM AS LEITURAS



Por que Usar Replicação?

Alta Disponibilidade: Garante que a aplicação continue operando mesmo com a falha de um servidor (failover automático).

Redundância de Dados: Protege contra a perda de dados, mantendo cópias em servidores diferentes.

Escalabilidade de Leitura: Permite distribuir a carga de leitura entre múltiplos servidores, melhorando a performance.

Manutenção sem Downtime: Possibilita atualizações e manutenções sem interromper o serviço.

A performance do MongoDB não é um acaso. Ela é o resultado de um conjunto de decisões de design focadas em resolver os gargalos de performance comuns em aplicações web modernas.

Os 5 Pilares da Performance:

- **O Modelo de Documentos:** A forma como os dados são guardados.
- **Uso Inteligente de Memória RAM:** Acesso rápido aos dados quentes.
- **Índices Flexíveis e Poderosos:** Ferramentas para buscas rápidas.
- **Arquitetura Escalável:** Distribuição da carga de trabalho.
- **Motor de Armazenamento Moderno (WiredTiger):** Eficiência no disco.

No MongoDB, dados que são acessados juntos são armazenados juntos em um único documento.

Impacto na Leitura:

Analogia: Imagine buscar informações de um cliente em uma biblioteca.

Banco Relacional (lento): Você pega o livro do cliente em uma estante, a ficha de pedidos dele em outra, e os comentários sobre os produtos em uma terceira. Você precisa correr pela biblioteca inteira e juntar tudo (uma operação de JOIN).

MongoDB (rápido): O perfil do cliente, seus pedidos e seus comentários estão todos na mesma "pasta". Você vai a um único lugar e pega tudo de uma vez. Uma única operação de leitura no disco.

Impacto na Escrita:

Para criar um novo pedido com vários itens, em vez de fazer INSERT em múltiplas tabelas (pedidos, itens_pedido, etc.), você insere um único documento, o que é uma operação atômica e muito rápida.

Uso Inteligente de Memória RAM (Working Set)

O MongoDB tenta manter os dados e os índices mais frequentemente acessados pela sua aplicação diretamente na memória RAM. Este conjunto de dados "quentes" é chamado de Working Set.

Analogia: É como um chef de cozinha que deixa os ingredientes mais usados (sal, azeite, pimenta) na bancada ao alcance da mão (a RAM), em vez de ter que ir buscá-los na despensa (o disco) a cada prato que prepara.

Impacto na Performance: Acessar dados na RAM é milhares de vezes mais rápido do que acessar dados no disco (mesmo em SSDs). Se o working set da sua aplicação couber na memória, suas leituras serão quase instantâneas.

Índices são como o índice remissivo de um livro. Em vez de ler o livro de 500 páginas do início ao fim para encontrar um termo, você vai ao índice, encontra o termo e vai direto para a página correta.

Como funciona: Um índice armazena uma cópia ordenada dos valores de um campo, com um ponteiro para o documento original.

Diferencial do MongoDB:

Suporta índices em qualquer campo de um documento, até mesmo em campos dentro de sub-documentos e arrays.

Possui tipos de índices especializados, como geoespaciais (para mapas) e de texto completo (para buscas).

Impacto na Performance: Uma consulta sem índice em uma coleção de bilhões de documentos (um collection scan) pode levar horas. A mesma consulta, usando um índice, pode retornar em milissegundos.

Índices são como o índice remissivo de um livro. Em vez de ler o livro de 500 páginas do início ao fim para encontrar um termo, você vai ao índice, encontra o termo e vai direto para a página correta.

Como funciona: Um índice armazena uma cópia ordenada dos valores de um campo, com um ponteiro para o documento original.

Diferencial do MongoDB:

Suporta índices em qualquer campo de um documento, até mesmo em campos dentro de sub-documentos e arrays.

Possui tipos de índices especializados, como geoespaciais (para mapas) e de texto completo (para buscas).

Impacto na Performance: Uma consulta sem índice em uma coleção de bilhões de documentos (um collection scan) pode levar horas. A mesma consulta, usando um índice, pode retornar em milissegundos.

Comparação: Relacional vs NoSQL

 Relacional: Tabelas com esquema rígido (SQL)

 NoSQL: Documentos, colunas, grafos
(esquema flexível)

 Joins vs Embedding (relacionamento entre dados)

 SQL: SELECT, JOIN, WHERE | NoSQL: JSON, consultas dinâmicas

 NoSQL ideal para grandes volumes e mudanças frequentes

Duvidas?



Referências

Documentação. Disponível em: <<http://www.mongodb.org/>>.

[Creative Commons](#). Processamento de Dados Massivos/Projeto e implementação de aplicações Big Data/Agregação eficiente de dados temporais. Disponível em:
http://pt.wikibooks.org/wiki/Processamento_de_Dados_Massivos/Projeto_e_implementação_de_aplicações_Big_Data/Agregação_eficiente_de_dados_temporais>.

Ianni, Vinicius. Introdução aos bancos de dados NoSQL. Disponível em: <<http://www.devmedia.com.br/introducao-aos-bancos-de-dados-nosql/26044#ixzz2O7Sed5P3>>.

Ianni, Vinicius. [Big Data: Algumas definições e, sim, serve também para o pequeno negócio!](#) Disponível em:
<<http://www.devmedia.com.br/space.asp?id=198225>>.

Lerner, Alberto. Introdução ao MongoDB – Direto da fonte! Disponível em: <<http://ensinar.wordpress.com/tag/mongodb/>>.

Lennon, Joe. Explore o MongoDB. Disponível em: <<http://www.ibm.com/developerworks/br/library/os-mongodb4/>>.

Nascimento, Jean3 Razões para usar MongoDB. Disponível em: <<http://imasters.com.br/artigo/18334/mongodb/3-razoes-para-usar-mongodb/>>.

Nascimento, Jean. Como utilizar selects com MongoDB. Disponível em: <<http://imasters.com.br/artigo/17308/mongodb/como->